

GITHUB BLOG · 2026.06.25

GitHub Copilot 智能体框架 性能与效率评测

同模型、同任务下，跨多个基准衡量任务完成率与 token 消耗——并保留 20+
模型的选择自由



Shibani Basava & Carlos Castro · agentic harness

核心结论

持平的完成率，更低的 token，开放的模型选择



完成率持平

多项基准上，与模型厂商自家框架的任务完成率相当，差异落在随机波动范围内。



更省 token

大多数配置下消耗更少 token，
Claude 系列单任务最高节省
65%。



不锁定模型

一套框架支持 **20+** 前沿模型，可按任务切换或交给 Auto 自动选择。

同样的任务完成度，更低的 token 成本，同时保留为每个任务挑选最合适模型的自由。

PART 01

框架是什么， 我们如何评测

agentic harness 的定位，与控制变量的基准方法

定义

什么是「智能体框架」 (agentic harness)

它是**编排层**——负责管理工具、上下文与 workflow，**独立于底层模型之外**。

模型负责“思考”，框架负责把每一轮的工具调用、上下文与记忆、限额与失败恢复编排起来。换模型，框架不变。

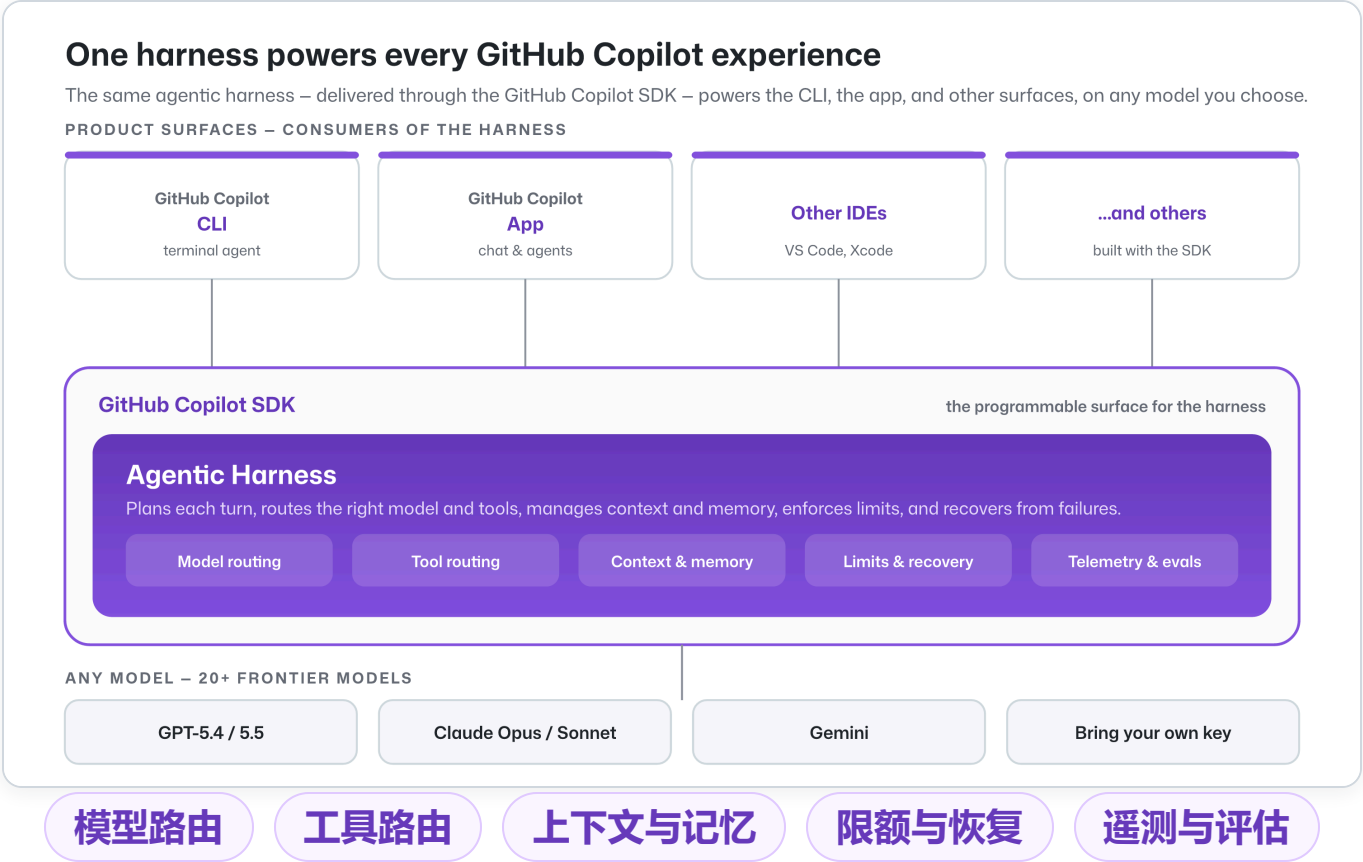
同一套框架，驱动 `Copilot CLI`、`App`、code review，以及 GitHub 与 Microsoft 的多处体验。

设计目标

- ◆ **快**：更短的端到端时延
- ◆ **省 token**：同样结果，更少消耗
- ◆ **可预测**：稳定、可复现的表现

架构

一套框架，驱动所有 GitHub Copilot 体验



框架经由 Copilot SDK 对外，承担上述五件事；模型可换，上层 CLI / App / IDE 体验不变。

我们如何用基准测试持续迭代

四类信号交叉验证

- ◆ **公共基准**：行业通用标准
- ◆ **内部基准**：取自 GitHub 与 Microsoft 真实代码库
- ◆ **真实使用指标**：来自线上日常使用
- ◆ **在线实验**：A/B 验证每次改动

尽可能控制每一个变量，让对比只反映框架本身的差异。

核心原则：控制变量

把 Copilot 框架与厂商自家框架放在同一条件下对比：

- ◆ 相同模型、相同基准任务
- ◆ 归一化上下文窗口与推理强度
- ◆ 统一工具选择与 `MCP` 配置

评测范围

四个模型 × 五项基准

参评模型

- Claude Sonnet 4.6
- Claude Opus 4.7
- GPT-5.4
- GPT-5.5

对照框架

- Copilot CLI 对比——
- ◆ Claude 模型 → Claude Code
- ◆ GPT 模型 → Codex CLI

基准	考察什么
SWE-bench Verified	500 个人工校验的开源 Python 修 bug 任务（行业标准）
SWE-bench Pro	更难的多步工程任务
SkillsBench	技能调用与任务解决（扩展性）
TerminalBench 2	终端 / 命令行工作流
Win-Hill	内部 Windows 容器任务（环境泛化）

PART 02

评测结果

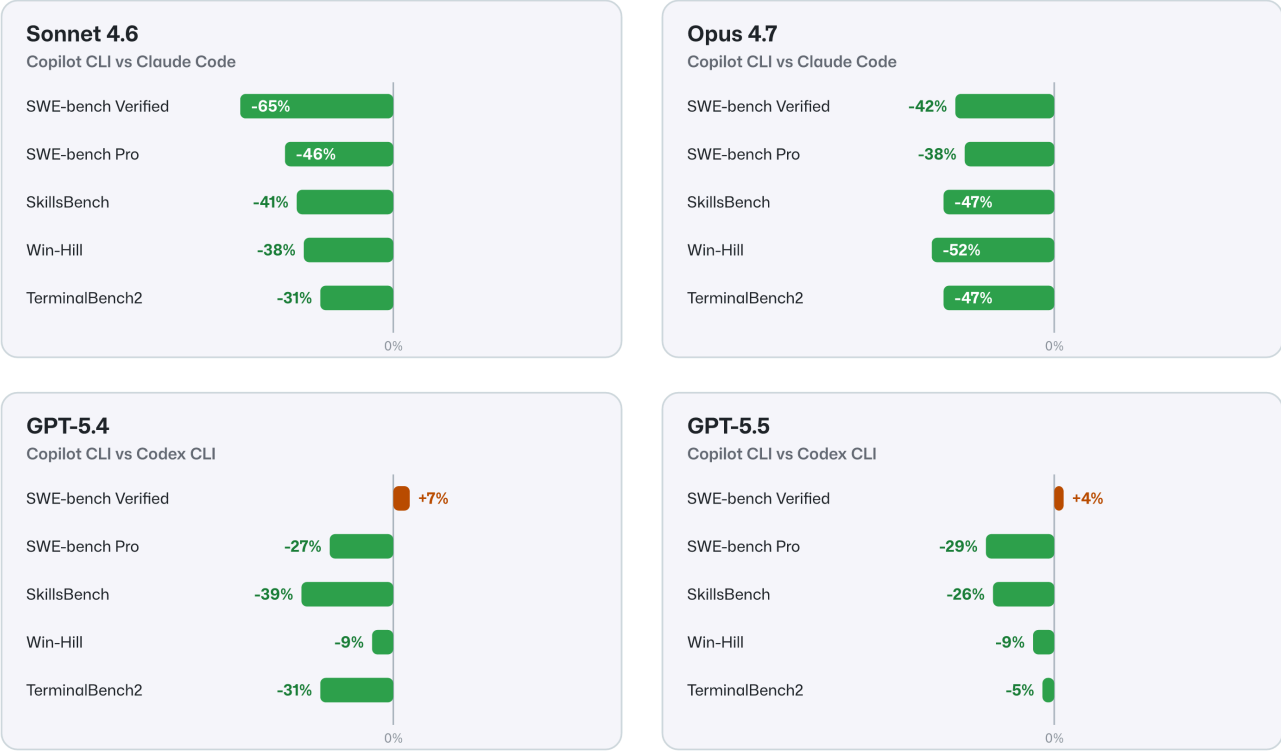
token 效率 · 任务完成率 · 成本与方差

同样的任务，更少的 token

Token efficiency – Copilot CLI vs. the model-vendor harnesses

Relative median (P50) tokens for the same task, same model and benchmark.
Negative = Copilot CLI uses fewer tokens (better) · 6/20 weekly suite · adjusted task set.

- Copilot CLI better (fewer tokens)
- Copilot CLI worse (more tokens)



条形为同一任务、同一模型下 token 中位数（P50）的相对差异；**负值 = Copilot 更省。**

- 👉 Claude 系列全面更省，最高 -65%
- 👉 GPT 系列多数更省（-5% ~ -39%）
- ⚠️ 唯一例外：SWE-bench Verified +7% / +4%

任务完成率

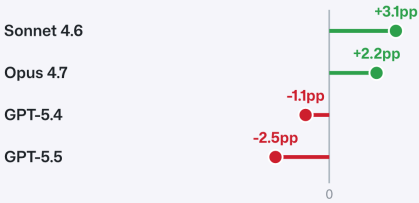
完成率与厂商框架基本持平

Task resolution – Copilot CLI vs. the model-vendor harnesses

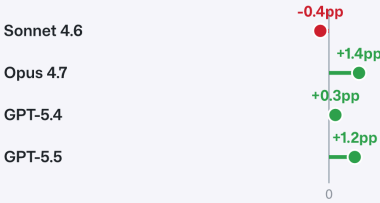
Difference in adjusted resolution rate (percentage points), same model and benchmark.
Positive = Copilot CLI resolves more tasks - 6/20 weekly suite.

Copilot CLI better (resolves more)
Copilot CLI worse (resolves fewer)
tie

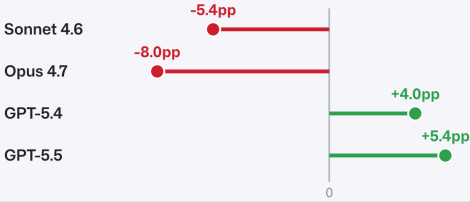
SWE-bench Verified



SWE-bench Pro



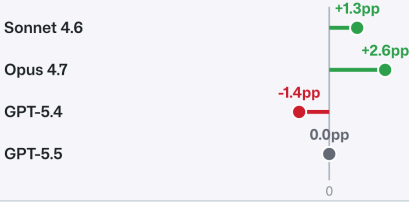
SkillsBench



Win-Hill



TerminalBench2



数值为完成率差异（百分点 pp）；正值 = Copilot 完成更多任务。

= 整体落在随机波动范围内

✓ Win-Hill：各模型持平或更优

↔ 逐项有来有回，如 SkillsBench

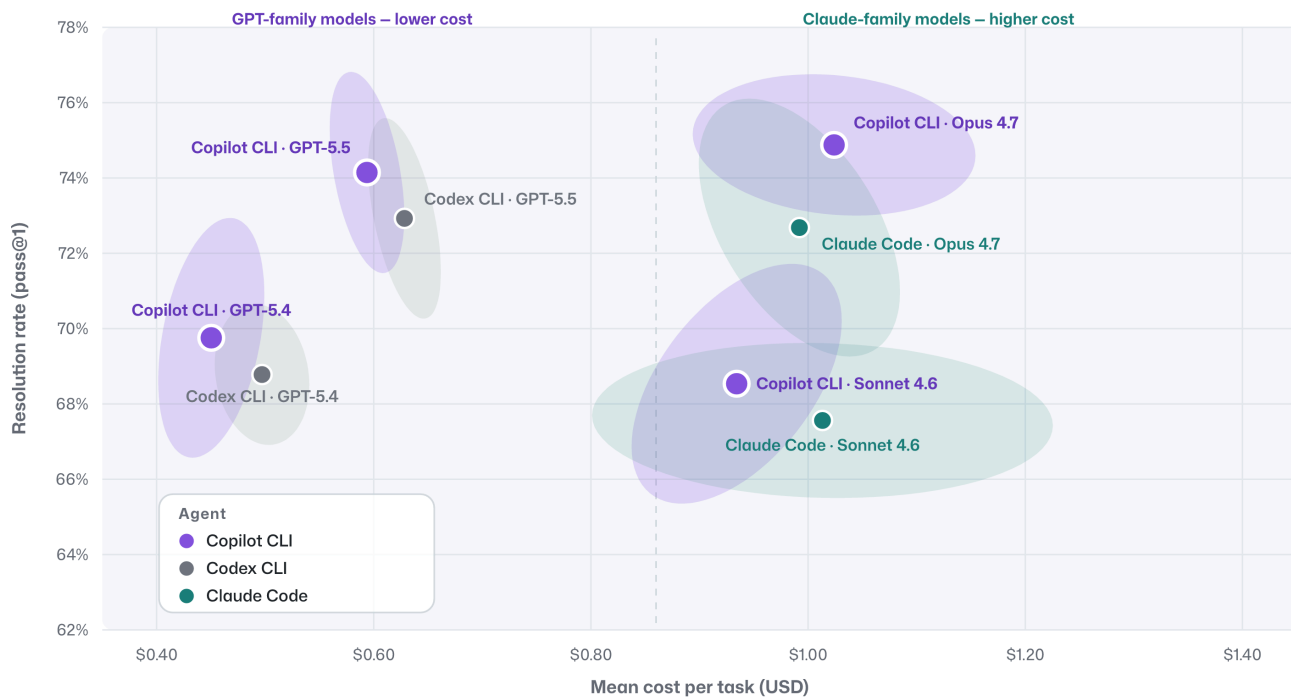
完成率持平 → 底层模型的**全部潜力**得以释放。

完成率 × 成本 × 方差：越靠左上越好

Resolution rate vs. cost per task

Each marker is one agent × model configuration (mean of 5 runs on Terminal-Bench 2). Shaded ellipse = $\pm 1\sigma$ run-to-run spread.

Up and to the left is better: higher resolution at lower cost.



① 持平或更优

紫色 Copilot CLI 点完成率不落后、成本不更高。

② 波动可控

每个配置 ≥ 5 次运行，椭圆为 $\pm 1\sigma$ ；越小越可复现。

③ 模型权衡

GPT 成本最低、性价比高；Opus 完成率最高但更贵——都能选。

PART 03

灵活性与结论

一套框架，多种模型

灵活性

一套框架，多种模型

20+ 前沿模型，自由切换

GPT-5.4 / 5.5

Claude Opus / Sonnet Gemini MAI

Bring-your-own-key

支持开源与本地模型；可按任务手动选择，或交给 **Auto** 自动挑选最合适的模型。



跨家族协作：Rubber Duck

让一个模型家族去**评审**另一个家族的产出——把单模型做不到的纠错，变成框架自带的能力，结果优于任一单模型。

结论

更省、不落后、不锁定

-65%

Claude 单任务 token
最高节省幅度

持平

任务完成率
对比厂商自家框架

20+

可选前沿模型
绝不锁定单一模型

GitHub Copilot 在多项配置下以更少 token 拿到与领先厂商框架相当的任务完成度，且凭借多模型架构，不把你锁死在单一模型上。

评测设置与重要说明

受控设置

- ◆ 2 小时超时，非交互式单轮执行
- ◆ 禁用 Web 工具，放开其余全部工具
- ◆ 归一化上下文窗口与推理强度（中）
- ◆ 指标统一为 `pass@1`；小型基准跑 5 次取最佳

读数前须知

- ◆ 受控实验，**不同于公开榜单成绩**
- ◆ 榜单常用更高推理强度与调优设置
- ◆ 排除基础设施异常与网络影响以求公平
- ◆ 基准只是**众多信号之一**

动手试试

用你选的模型，跑一遍日常任务



Copilot CLI

终端里的智能体



Copilot App

对话与智能体



Copilot SDK

把框架编进你的工具

同一套 agentic harness 驱动以上全部体验——**在你的环境里**对比模型与策略。